

C Programming: Complete Guide

Table of Contents

1. Control Structures

- Selection Structures
- Looping Structures

2. Character Input/Output

- Character I/O Functions
- String I/O Functions
- Formatted I/O

3. Coding Standards and Guidelines

- Naming Conventions
- Indentation and Formatting
- Comments and Documentation
- Error Handling

4. Arrays - 1D, Initialization and Traversal

- Declaration and Initialization
- Accessing and Modifying Elements
- Array Traversal Techniques

5. Arrays - 2D

- Declaration and Initialization
- Accessing and Modifying Elements
- Common 2D Array Operations

6. Pointers

- Pointer Basics
- Pointer Arithmetic
- Pointers and Arrays

7. Functions

- Function Declaration and Definition
- Parameter Passing
- Return Values

8. Arrays and Functions

- Passing Arrays to Functions

- Returning Arrays from Functions

Control Structures

Control structures in C programming determine the flow of program execution. They fall into two main categories: selection structures and looping structures.

Selection Structures

Selection structures allow a program to choose different execution paths based on conditions.

if Statement

The `if` statement executes a block of code if a specified condition is true.

```
c
#include <stdio.h>

int main() {
    int age = 20;

    if (age >= 18) {
        printf("You are an adult.\n");
    }

    return 0;
}
```

if-else Statement

The `if-else` statement provides an alternative path when the condition is false.

c

```
#include <stdio.h>

int main() {
    int number = 7;

    if (number % 2 == 0) {
        printf("%d is even.\n", number);
    } else {
        printf("%d is odd.\n", number);
    }

    return 0;
}
```

if-else if-else Ladder

For multiple conditions, the `if-else if-else` ladder can be used.

c

```
#include <stdio.h>

int main() {
    int score = 85;

    if (score >= 90) {
        printf("Grade: A\n");
    } else if (score >= 80) {
        printf("Grade: B\n");
    } else if (score >= 70) {
        printf("Grade: C\n");
    } else if (score >= 60) {
        printf("Grade: D\n");
    } else {
        printf("Grade: F\n");
    }

    return 0;
}
```

Nested if Statements

An `if` statement can be nested inside another `if` or `else` block.

c

```
#include <stdio.h>

int main() {
    int age = 25;
    int hasLicense = 1; // 1 for true, 0 for false

    if (age >= 18) {
        printf("Age requirement met.\n");

        if (hasLicense) {
            printf("You can drive.\n");
        } else {
            printf("You need a license to drive.\n");
        }
    } else {
        printf("You are too young to drive.\n");
    }

    return 0;
}
```

switch Statement

The `switch` statement provides a cleaner way to handle multiple conditions based on a single value.

c

```
#include <stdio.h>

int main() {
    char grade = 'B';

    switch (grade) {
        case 'A':
            printf("Excellent!\n");
            break;
        case 'B':
            printf("Good job!\n");
            break;
        case 'C':
            printf("Satisfactory.\n");
            break;
        case 'D':
            printf("Needs improvement.\n");
            break;
        case 'F':
            printf("Failed.\n");
            break;
        default:
            printf("Invalid grade.\n");
    }

    return 0;
}
```

Important notes about `switch`:

- Each `case` should end with a `break` statement to prevent fall-through.
- The `default` case handles any value not matched by the cases.
- The expression in the `switch` statement must be integral (int, char, enum).

Conditional Operator

The conditional operator (`? :`) provides a compact way to express simple if-else logic.

c

```
#include <stdio.h>

int main() {
    int age = 20;

    // Using conditional operator
    printf("You are %s.\n", (age >= 18) ? "an adult" : "a minor");

    // Equivalent if-else
    if (age >= 18) {
        printf("You are an adult.\n");
    } else {
        printf("You are a minor.\n");
    }

    return 0;
}
```

Looping Structures

Looping structures allow a program to repeat a block of code multiple times.

for Loop

The `for` loop is typically used when the number of iterations is known beforehand.

c

```
#include <stdio.h>

int main() {
    // Print numbers from 1 to 5
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    printf("\n");

    // Calculate the sum of numbers from 1 to 10
    int sum = 0;
    for (int i = 1; i <= 10; i++) {
        sum += i;
    }
    printf("Sum: %d\n", sum);

    return 0;
}
```

The `for` loop has three parts:

1. **Initialization:** Executed once before the loop starts.
2. **Condition:** Checked before each iteration; if false, the loop terminates.
3. **Update:** Executed after each iteration.

while Loop

The `while` loop repeats as long as a condition is true.

c

```
#include <stdio.h>

int main() {
    int count = 1;

    while (count <= 5) {
        printf("%d ", count);
        count++;
    }
    printf("\n");

    // Reading until a specific input
    int number;
    printf("Enter a number (0 to quit): ");
    scanf("%d", &number);

    while (number != 0) {
        printf("You entered: %d\n", number);
        printf("Enter another number (0 to quit): ");
        scanf("%d", &number);
    }

    return 0;
}
```

do-while Loop

The `do-while` loop is similar to the `while` loop, but it guarantees that the loop body executes at least once.

c

```
#include <stdio.h>

int main() {
    int count = 1;

    do {
        printf("%d ", count);
        count++;
    } while (count <= 5);
    printf("\n");

    // Even if the condition is initially false, the loop executes once
    int number = 10;

    do {
        printf("This will print once even though number > 5\n");
    } while (number <= 5);

    return 0;
}
```

break and continue Statements

- `break`: Exits the innermost loop immediately.
- `continue`: Skips the rest of the current iteration and moves to the next iteration.

c

```
#include <stdio.h>

int main() {
    // Using break to exit the loop early
    printf("Break example:\n");
    for (int i = 1; i <= 10; i++) {
        if (i == 6) {
            break; // Exit the loop when i is 6
        }
        printf("%d ", i);
    }
    printf("\n");

    // Using continue to skip specific iterations
    printf("Continue example:\n");
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue; // Skip even numbers
        }
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

Nested Loops

Loops can be nested within other loops to handle more complex patterns.

c

```
#include <stdio.h>

int main() {
    // Print a pattern
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Print a multiplication table
    printf("\nMultiplication Table:\n");
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            printf("%d\t", i * j);
        }
        printf("\n");
    }

    return 0;
}
```

Character Input/Output

Character I/O operations allow a program to read and write characters and strings.

Character I/O Functions

getchar() and putchar()

`getchar()` reads a single character from stdin, and `putchar()` writes a single character to stdout.

c

```
#include <stdio.h>

int main() {
    printf("Enter a character: ");
    char ch = getchar();

    printf("You entered: ");
    putchar(ch);
    printf("\n");

    return 0;
}
```

fgetc() and fputc()

These functions work like `getchar()` and `putchar()` but allow specifying the stream.

c

```
#include <stdio.h>

int main() {
    // Reading from stdin (equivalent to getchar())
    printf("Enter a character: ");
    char ch = fgetc(stdin);

    // Writing to stdout (equivalent to putchar())
    printf("You entered: ");
    fputc(ch, stdout);
    printf("\n");

    // File I/O
    FILE *file = fopen("char.txt", "w");
    if (file != NULL) {
        fputc('A', file);
        fputc('B', file);
        fputc('C', file);
        fclose(file);

        printf("Characters written to file.\n");
    }

    return 0;
}
```

String I/O Functions

gets() and puts() (Deprecated)

`gets()` reads a line from stdin until a newline or EOF, and `puts()` writes a string to stdout followed by a newline.

c

```
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter a string: ");
    // gets(str); // WARNING: gets() is deprecated due to buffer overflow risk

    // Use fgets() instead
    fgets(str, sizeof(str), stdin);

    printf("You entered: ");
    puts(str);

    return 0;
}
```

fgets() and fputs()

These functions provide safer string I/O and allow specifying the stream.

c

```
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // Remove the newline character if present
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == '\n') {
            str[i] = '\0';
            break;
        }
    }

    printf("You entered: ");
    fputs(str, stdout);
    printf("\n");

    // File I/O
    FILE *file = fopen("string.txt", "w");
    if (file != NULL) {
        fputs("Hello, World!", file);
        fclose(file);

        printf("String written to file.\n");
    }

    return 0;
}
```

Formatted I/O

printf() and scanf()

`printf()` formats and writes data to stdout, and `scanf()` reads formatted data from stdin.

c

```
#include <stdio.h>

int main() {
    int age;
    float height;
    char name[50];

    printf("Enter your name: ");
    scanf("%49s", name); // Limit to 49 characters to avoid buffer overflow

    printf("Enter your age and height (in meters): ");
    scanf("%d %f", &age, &height);

    printf("\nYour details:\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f meters\n", height);

    return 0;
}
```

fprintf() and fscanf()

These functions work like `printf()` and `scanf()` but allow specifying the stream.

c

```
#include <stdio.h>

int main() {
    int id, age;
    char name[50];

    // Writing to a file
    FILE *file = fopen("data.txt", "w");
    if (file != NULL) {
        fprintf(file, "%d %s %d\n", 1, "John", 25);
        fprintf(file, "%d %s %d\n", 2, "Alice", 30);
        fclose(file);

        printf("Data written to file.\n");
    }

    // Reading from the file
    file = fopen("data.txt", "r");
    if (file != NULL) {
        printf("\nReading data from file:\n");

        while (fscanf(file, "%d %s %d", &id, name, &age) == 3) {
            printf("ID: %d, Name: %s, Age: %d\n", id, name, age);
        }

        fclose(file);
    }

    return 0;
}
```

sprintf() and sscanf()

These functions allow formatting and parsing strings in memory.

c

```
#include <stdio.h>

int main() {
    char buffer[100];
    int age = 25;
    float height = 1.75;

    // Format data into a string
    sprintf(buffer, "Age: %d, Height: %.2f meters", age, height);
    printf("Formatted string: %s\n", buffer);

    // Parse data from a string
    char str[] = "ID: 123 Score: 95.5";
    int id;
    float score;

    sscanf(str, "ID: %d Score: %f", &id, &score);
    printf("Parsed data - ID: %d, Score: %.1f\n", id, score);

    return 0;
}
```

Coding Standards and Guidelines

Coding standards ensure code readability, maintainability, and consistency across projects.

Naming Conventions

Variable and Function Names

- Use meaningful names that describe the purpose.
- Follow a consistent case style (e.g., camelCase or snake_case).

c

```
// Good practice
int studentCount;
float averageScore;
void calculateTotalSum();

// Bad practice
int a;
float x;
void calc();
```

Constants and Macros

- Use uppercase letters with underscores for constants and macros.

c

```
#define MAX_BUFFER_SIZE 1024
```

```
#define PI 3.14159
```

```
const int MIN_AGE = 18;
```

Indentation and Formatting

Indentation

- Use consistent indentation (typically 2 or 4 spaces).
- Align braces properly.

c

```
// Consistent indentation example
```

```
void processData(int value) {  
    if (value > 0) {  
        printf("Positive value\n");  
        value *= 2;  
    } else {  
        printf("Non-positive value\n");  
        value = 0;  
    }  
  
    printf("Final value: %d\n", value);  
}
```

Line Length

- Keep lines reasonably short (e.g., 80-120 characters).
- Break long lines logically.

c

```
// Long line broken into multiple lines
int result = functionWithLongName(parameter1, parameter2,
                                   parameter3, parameter4);

// Long condition broken into multiple lines
if (condition1 &&
    condition2 &&
    condition3) {
    // Code block
}
```

Comments and Documentation

Function Comments

- Document function purpose, parameters, and return values.

c

```
/**
 * Calculates the average of an array of integers.
 *
 * @param arr The input array
 * @param size The size of the array
 * @return The average of the elements
 */
float calculateAverage(int arr[], int size) {
    if (size <= 0) {
        return 0.0f;
    }

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }

    return (float)sum / size;
}
```

Inline Comments

- Use inline comments to explain complex logic.

c

```
void processTransaction(Transaction *t) {  
    // Skip inactive transactions  
    if (!t->isActive) {  
        return;  
    }  
  
    // Apply discount for premium customers  
    if (t->customerType == PREMIUM) {  
        t->amount *= 0.9; // 10% discount  
    }  
  
    // Calculate tax based on region  
    calculateRegionalTax(t);  
}
```

Error Handling

Return Codes

- Use consistent return codes for error handling.

c

```
#define SUCCESS 0  
#define ERROR_FILE_NOT_FOUND -1  
#define ERROR_INVALID_INPUT -2  
  
int processFile(const char *filename) {  
    FILE *file = fopen(filename, "r");  
    if (file == NULL) {  
        return ERROR_FILE_NOT_FOUND;  
    }  
  
    // Process the file  
    fclose(file);  
    return SUCCESS;  
}
```

Input Validation

- Always validate input before processing.

c

```
int divide(int a, int b, int *result) {
    if (b == 0) {
        return -1; // Error: division by zero
    }

    *result = a / b;
    return 0; // Success
}

int main() {
    int a = 10, b = 0, result;

    if (divide(a, b, &result) == 0) {
        printf("Result: %d\n", result);
    } else {
        printf("Error: Division by zero\n");
    }

    return 0;
}
```

Arrays - 1D, Initialization and Traversal

Arrays are collections of elements of the same type stored in contiguous memory locations.

Declaration and Initialization

Array Declaration

c

```
// Declaration without initialization
int numbers[5]; // Array of 5 integers

// Declaration with size determined by the compiler
int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

Array Initialization

c

```
#include <stdio.h>

int main() {
    // Initialize at declaration
    int scores[5] = {95, 88, 76, 90, 82};

    // Partial initialization (remaining elements are set to 0)
    int values[5] = {10, 20}; // Equivalent to {10, 20, 0, 0, 0}

    // Initialize all elements to zero
    int zeros[10] = {0};

    // Initialize after declaration
    int grades[3];
    grades[0] = 85;
    grades[1] = 92;
    grades[2] = 78;

    return 0;
}
```

Accessing and Modifying Elements

c

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    // Accessing elements
    printf("First element: %d\n", numbers[0]);
    printf("Third element: %d\n", numbers[2]);

    // Modifying elements
    numbers[1] = 25;
    numbers[4] = 55;

    // Displaying all elements
    printf("Updated array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}
```

Array Traversal Techniques

Using for Loop

c

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int sum = 0;

    // Simple traversal
    printf("Array elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    // Traversal with calculation
    for (int i = 0; i < 5; i++) {
        sum += numbers[i];
    }
    printf("Sum of elements: %d\n", sum);

    // Reverse traversal
    printf("Array in reverse: ");
    for (int i = 4; i >= 0; i--) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}
```

Using while Loop

c

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int i = 0;

    printf("Array elements (using while): ");
    while (i < 5) {
        printf("%d ", numbers[i]);
        i++;
    }
    printf("\n");

    return 0;
}
```

Using Pointers

c

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int *ptr = numbers; // Pointer to the first element

    printf("Array elements (using pointer): ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");

    // Alternative pointer traversal
    printf("Array elements (using pointer arithmetic): ");
    for (int *p = numbers; p < numbers + 5; p++) {
        printf("%d ", *p);
    }
    printf("\n");

    return 0;
}
```

Arrays - 2D

Two-dimensional arrays represent a table of elements with rows and columns.

2D Declaration and Initialization

c

```
#include <stdio.h>

int main() {
    // Declaration with initialization
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // Partial initialization
    int grid[2][3] = {
        {1, 2, 3},    // Row 0
        {4, 5}        // Row 1, column 2 defaults to 0
    };

    // Row-by-row initialization (same as matrix above)
    int another[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};

    // Initialize all elements to zero
    int zeros[3][3] = {{0}};

    return 0;
}
```

2D Accessing and Modifying Elements

c

```
#include <stdio.h>

int main() {
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Accessing elements
    printf("Element at row 1, column 2: %d\n", matrix[1][2]); // Output: 6

    // Modifying elements
    matrix[0][0] = 10;
    matrix[2][2] = 99;

    // Display the modified matrix
    printf("Modified matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Common 2D Array Operations

Matrix Addition

C

```
#include <stdio.h>

int main() {
    int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int B[2][3] = {{7, 8, 9}, {10, 11, 12}};
    int C[2][3]; // To store the result

    // Add matrices A and B
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    // Display the result
    printf("Matrix Addition Result:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%2d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Matrix Multiplication

C

```
#include <stdio.h>

int main() {
    int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int B[3][2] = {{7, 8}, {9, 10}, {11, 12}};
    int C[2][2] = {0}; // To store the result, initialized to zeros

    // Multiply matrices A and B
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 3; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    // Display the result
    printf("Matrix Multiplication Result:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%3d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Finding the Sum of Diagonals

C

```
#include <stdio.h>

int main() {
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int mainDiagonalSum = 0;
    int antiDiagonalSum = 0;

    for (int i = 0; i < 3; i++) {
        mainDiagonalSum += matrix[i][i];           // Main diagonal
        antiDiagonalSum += matrix[i][3 - 1 - i];   // Anti-diagonal
    }

    printf("Main Diagonal Sum: %d\n", mainDiagonalSum);
    printf("Anti-Diagonal Sum: %d\n", antiDiagonalSum);

    return 0;
}
```

Pointers

Pointers are variables that store memory addresses.

Pointer Basics

c

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr; // Declare a pointer to an integer

    ptr = &num; // Assign the address of num to ptr

    printf("Value of num: %d\n", num);
    printf("Address of num: %p\n", &num);
    printf("Value of ptr (address it points to): %p\n", ptr);
    printf("Value at the address stored in ptr: %d\n", *ptr);

    // Modifying the value using pointer
    *ptr = 20;
    printf("New value of num: %d\n", num);

    return 0;
}
```

Pointer Arithmetic

c

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int *ptr = numbers; // Points to the first element

    printf("Array traversal using pointer arithmetic:\n");

    // Accessing elements using pointer arithmetic
    printf("Element 0: %d\n", *ptr);           // 10
    printf("Element 1: %d\n", *(ptr + 1)); // 20
    printf("Element 2: %d\n", *(ptr + 2)); // 30

    // Moving the pointer
    ptr++; // Now points to the second element
    printf("After ptr++, pointer points to: %d\n", *ptr); // 20

    ptr += 2; // Now points to the fourth element
    printf("After ptr += 2, pointer points to: %d\n", *ptr); // 40

    ptr--; // Now points back to the third element
    printf("After ptr--, pointer points to: %d\n", *ptr); // 30

    // Comparing pointers
    int *p1 = &numbers[1];
    int *p2 = &numbers[3];

    printf("Pointer comparison: p2 - p1 = %ld elements\n", p2 - p1);

    return 0;
}
```

Pointers and Arrays

c

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    // Array name as a pointer
    printf("numbers is a pointer to the first element: %p\n", numbers);
    printf("&numbers[0] gives the same address: %p\n", &numbers[0]);

    // Equivalence of array indexing and pointer arithmetic
    printf("numbers[2] = %d\n", numbers[2]);
    printf("*(numbers + 2) = %d\n", *(numbers + 2));

    // These two expressions are equivalent
    printf("3[numbers] = %d\n", 3[numbers]);          // Unusual but valid
    printf("*(3 + numbers) = %d\n", *(3 + numbers));

    return 0;
}
```

Functions

Functions are self-contained blocks of code that perform specific tasks.

Function Declaration and Definition

c

```
#include <stdio.h>

// Function declaration (prototype)
int add(int a, int b);
void greet(char name[]);
float calculateAverage(int values[], int size);

int main() {
    int result = add(5, 3);
    printf("5 + 3 = %d\n", result);

    greet("John");

    int scores[] =
```